

Exam Date & Time: 04-Dec-2024 (02:30 PM - 05:30 PM)



MANIPAL ACADEMY OF HIGHER EDUCATION

MIT BLR - BTech VII Semester - End Semester Examination - Nov / Dec -2024

SOFTWARE TESTING AND ANALYSIS [CSE-4076]

Marks: 50

Duration: 180 mins.

PART-A

Answer all the questions.

- 1) Compare system and acceptance testing using a suitable example. (3)
- A)
- B) Identify the purpose of Test Oracle using a suitable example (2)
- C) For the given program, draw the data-flow graph, list the Definitions, P-Use and C-Use of all the variables used and list the DU pairs. Also give test cases to cover All-Definitions. (5)
- ```
main()
{
float bs, gs, da, hra = 0;
printf("Enter basic salary");
scanf("%f", &bs);
if(bs < 1500)
{
hra = bs * 10/100;
da = bs * 90/100;
}
else
{
hra = 500;
da = bs * 98/100;
}
gs = bs + hra + da;
printf("Gross Salary = Rs. %f", gs);
}
```
- 2) A university is admitting students in a professional course subject to the following conditions: (4)
- A) (a) Marks in Java  $\geq 70$   
(b) Marks in C++  $\geq 60$   
(c) Marks in OOAD  $\geq 60$   
(d) Total in all three subjects  $\geq 220$  OR Total in Java and C++  $\geq 150$   
If the aggregate mark of an eligible candidate is more than 240, he will be eligible for a

scholarship course, otherwise he will be eligible for normal course.

The program reads the marks in the three subjects and generates the following outputs:

- (i) Not eligible
- (ii) Eligible for scholarship course
- (iii) Eligible for normal course

Identify the test cases for weak normal and weak robust equivalence class testing.

B) Consider the following program:

```
main ()
{
int a,b, div, modulo, percentage;
scanf("%d %d %d", &a, &b);
div=division(a,b,c);
percentage=calPercen(a,b,c);
modulo=calModulo(a,b,c);
printf("%d %d %d", div, modulo, percentage);
}
division (int x, int y){
int d;
d=x/y; return(d); }
```

(3)

Consider the following two scenarios.

- i. Suppose, the main ( ) module is not ready for the testing of division ( ) module. Apply suitable testing strategy to test the whole system in such a scenario.
- ii. Suppose modules calPercen ( ) and calModulo ( ) are not ready when called in main(). Apply suitable testing strategy to test the whole system in such a scenario.

C) Consider the following program. Draw the Control Flow Graph.

```
1. begin
2 float A,B,C,D, x1, x2
3 boolean is_complex
4 input(A,B,C)
5 D = B*B -4*A*C
6 if D < 0.0
7 then is_complex = true
8 else is_complex=false
9 end if
10 if not is_complex
11 then x1 = (-B + sqrt(D)) / (2.0*A)
12 x2 = (-B - sqrt(D)) / (2.0*A)
13 end if
14 output(x1,x2)
15 end
```

(3)

Design test cases to achieve maximum coverage of branch and condition coverage.

3) List the benefits of dataflow coverage criteria over structural coverage criteria in (2)

## Integration Testing.

- A)
- B) Analyse the purpose of YO-YO graph with a suitable example. (3)
- C) Consider the following program. (5)

```

public class PrintPrimes
{
 public static void main (String []argv)
 { // Driver method for printPrimes
 // Read an integer from standard input, call printPrimes()

 int integer = 0;
 integer = Integer.parseInt (argv[0]);
 printPrimes (integer);
 }
 private static void printPrimes (int n)
 {
 int curPrime; // Value currently considered for primeness
 int numPrimes; // Number of primes found so far.
 boolean isPrime; // Is curPrime prime?
 int [] primes = new int [100]; // The list of prime numbers.

 // Initialize 2 into the list of primes.
 primes [0] = 2;
 numPrimes = 1;
 curPrime = 2;
 while (numPrimes < n)
 {
 curPrime++; // next number to consider ...
 isPrime = true;
 for (int i = 0; i <= numPrimes-1; i++)
 { // for each previous prime.
 if (isDivisible (primes[i], curPrime))
 { // Found a divisor, curPrime is not prime.
 isPrime = false;
 break; // out of loop through primes.
 }
 }
 if (isPrime)
 { // save it!
 primes[numPrimes] = curPrime;
 numPrimes++;
 }
 } // End while
 // Print all the primes out.
 for (int i = 0; i <= numPrimes-1; i++)
 {
 System.out.println ("Prime: " + primes[i]);
 }
 }
 }

```

```

} // end printPrimes

private static boolean isDivisible (int i, int j)
{
 if (j%i == 0)
 return true;
 else
 return false;
}
}

```

(i) Identify Last Defs and First Uses of different variables.

(ii) Identify Coupling du-pairs in triplet form (method name, variable name, statement number)

4) Analyse Test Prioritization Technique with a suitable example.

(3)

A)

B)

The inventory management system at Hoosier Burger is designed to handle seasonal changes in business demand based on whether it is the Academic Season, Summer Season, or Holiday Season. The inventory ordering process varies for perishable and nonperishable items based on the season and the type of item. Here's how the process works in each season:

Academic Season: If an item is perishable, such as meat, vegetables, milk etc., the inventory system has a standing daily order with a local supplier stating that a pre-specified amount of food is delivered each weekday for that day's use and a standing weekend order with a local supplier each Saturday for weekend use. If an item is nonperishable, such as straws, cups and napkins, a minimum order quantity is placed when the stock on hand reaches a certain predetermined minimum reorder quantity.

(5)

Summer Season: The standing orders with all their suppliers are reduced by specific amounts during the summer i.e. summer reduction.

Holiday Season: The standing orders with all their suppliers are reduced by specific amounts during the holiday break i.e. Holiday reduction.

Design test cases for this system using decision table testing.

C)

Compare branch coverage and condition coverage using a suitable example.

(2)

5) Consider the following Program:

(5)

A)

```

function MAX(M< N : INTEGER)
return Integer is
begin
 if M>N Then
 return M;
 else
 return N
 end if;
end function MAX;

```

```
end if
end MAX
```

Consider five mutants of the above program by replacing ">" operator in if statement by (<, ≤, ≥, =, or ≠). Generate 2 test cases to achieve 100% mutation score adequacy.

- B) A program reads the data of employees in a company by taking the following inputs and prints them:  
Name of Employee (Max. 15 valid characters A-Z, a-z, space)  
Employee ID (exactly 10 characters) (5)  
Designation (up to 20 characters)

Design test cases for this program using normal and robust boundary value testing.

-----End-----